FIAP • Desafio de Testes Automatizados

Cidades ESG Inteligentes

BDD, testes de API e contrato com JSON Schema

Integrante
Sidney — RM: 562855

Repositório
https://github.com/jrpietrafesa/cidades-inteligentes
Branch: testes-automatizados

Stack de testes
Cucumber 7.20 • Gherkin • RestAssured • JSON Schema • JUnit 5
Docker Compose • PostgreSQL 16 • GitHub Actions

Data
Novembro de 2026

1. Visão geral

O projeto é a API do desafio anterior: um serviço Java Spring Boot que monitora o consumo de energia de edifícios públicos. Cada edifício tem uma meta diária em kWh. As medições são somadas por dia e, quando a soma passa da meta, a API abre um alerta. O endpoint de indicadores consolida consumo, emissão estimada de CO_2 e alertas.

Nesta atividade foi criada uma suíte que valida esse comportamento pela interface pública da API, com cenários escritos em português e executáveis.

31	**10 de 10**	**10**	**2**
cenários BDD, 16 positivos e 15 negativos	endpoints testados	contratos JSON Schema	ambientes: H2 e PostgreSQL

Atendimento aos requisitos

Requisito	Como foi atendido	Seção
Mínimo de 3 cenários BDD, positivos e negativos	31 cenários em Gherkin, sendo 16 positivos e 15 negativos	3
Testes de todas as APIs	10 endpoints cobertos com RestAssured	4
Validação de status code	Em todos os cenários, incluindo 200, 201, 400 e 404	4
Validação do corpo JSON	Valores de campos, listas, ordenação e cálculos	3 e 4
Contrato com JSON Schema	10 schemas com campos obrigatórios e tipos	4
Execução simples e documentada	Um comando local, um comando com Docker e pipeline no GitHub Actions	6
Evidências de execução	Relatórios, logs e dados das execuções reais	7 e 8

Relação com ESG

- **Ambiental:** os cenários `@ambiental` garantem o cálculo correto da meta de consumo, dos alertas e da emissão estimada de CO_2.
- **Governança:** os cenários `@governanca` garantem a rastreabilidade de cada alerta (consumo, meta, data e status), o histórico de medições e a rejeição de dados inválidos.

O tema se enquadra nos exemplos 1 (eficiência energética) e 4 (governança e compliance ambiental) do enunciado.

2. Estratégia de testes

Ferramenta	Papel
Gherkin	Linguagem dos cenários, em português (`# language: pt`)
Cucumber 7.20	Executa os arquivos `.feature` e gera relatórios HTML, JSON e XML
RestAssured	Faz as chamadas HTTP reais e lê as respostas
JSON Schema Validator	Valida o contrato de cada resposta
JUnit 5 e AssertJ	Executam a suíte e fazem as comparações
Docker Compose	Sobe a API, o PostgreSQL e o container de testes
GitHub Actions	Executa tudo a cada push e guarda os relatórios

Dois ambientes de execução

A classe `AmbienteDeTeste` escolhe o alvo pela variável `BASE_URL`:

- **Sem `BASE_URL`:** sobe a aplicação real em porta aleatória, com H2 em memória. É a opção mais rápida.
- **Com `BASE_URL`:** testa uma API já em execução. O Docker Compose usa essa opção para testar a imagem da aplicação com PostgreSQL 16.

Os mesmos cenários rodam nos dois casos. Para funcionar em um banco que já tem dados, cada cenário cria seu próprio edifício e os indicadores são validados pela variação, comparando os valores antes e depois.

Organização dos arquivos

Estrutura dos testes

```
src/test/
├── java/br/com/fiap/esg/bdd/
│   ├── BddApiTest.java               # executa os cenários
│   ├── AmbienteDeTeste.java          # define a API testada (H2 ou BASE_URL)
│   ├── ContextoCenario.java          # estado compartilhado do cenário
│   ├── Ganchos.java                  # início, fim e registro de falhas
│   ├── MonitoramentoSteps.java       # passos de negócio
│   ├── RespostaSteps.java            # status, corpo JSON e contrato
│   └── ContratoJsonSchemaTest.java   # prova que os contratos reprovam erros
└── resources/
    ├── features/                     # 5 arquivos .feature
    ├── schemas/                      # 10 contratos JSON Schema
    └── junit-platform.properties     # configuração do Cucumber
Dockerfile.tests · docker-compose.test.yml · scripts/run-tests.sh
.github/workflows/testes-automatizados.yml
```

3. Cenários de teste

Lista gerada do relatório de execução, com o número de passos de cada cenário. O texto completo está no Apêndice.

Área	Cenário	Tipo	Pilar ESG	Passos
Alertas	Consultar alerta aberto com dados de rastreabilidade	Positivo	Governança	4
Alertas	Nova medição no mesmo dia atualiza o alerta existente	Positivo	Governança	4
Alertas	Resolver um alerta aberto	Positivo	Governança	6
Alertas	Resolver um alerta que não existe	Negativo	–	4
Edifícios	Cadastrar edifício com meta ESG válida	Positivo	Ambiental	7
Edifícios	Consultar um edifício cadastrado	Positivo	–	5
Edifícios	Listar os edifícios monitorados	Positivo	–	5
Edifícios	Rejeitar cadastro inválido: nome em branco	Negativo	Governança	4
Edifícios	Rejeitar cadastro inválido: cidade em branco	Negativo	Governança	4
Edifícios	Rejeitar cadastro inválido: tipo em branco	Negativo	Governança	4
Edifícios	Rejeitar cadastro inválido: meta zerada	Negativo	Governança	4
Edifícios	Rejeitar cadastro inválido: meta negativa	Negativo	Governança	4
Edifícios	Rejeitar cadastro inválido: meta ausente	Negativo	Governança	4
Edifícios	Rejeitar corpo com JSON malformado	Negativo	–	2
Edifícios	Consultar um edifício que não existe	Negativo	–	4
Edifícios	Consultar um edifício com identificador inválido	Negativo	–	2
Indicadores	Indicadores refletem uma medição acima da meta	Positivo	Ambiental	11
Indicadores	Resolver alerta atualiza os indicadores	Positivo	Governança	8
Medições	Consumo dentro da meta não gera alerta	Positivo	Ambiental	6
Medições	Consumo exatamente igual à meta não gera alerta	Positivo	Ambiental	3
Medições	Consumo acumulado do dia acima da meta gera alerta	Positivo	Ambiental	5
Medições	Medições de dias diferentes não se somam	Positivo	Ambiental	4
Medições	Medição sem data usa o horário atual	Positivo	–	4
Medições	Listar o histórico de medições do edifício	Positivo	Governança	7
Medições	Rejeitar medição inválida: consumo zerado	Negativo	Governança	4
Medições	Rejeitar medição inválida: consumo negativo	Negativo	Governança	4
Medições	Rejeitar medição inválida: consumo ausente	Negativo	Governança	4
Medições	Registrar medição em edifício que não existe	Negativo	–	3
Medições	Listar medições de edifício que não existe	Negativo	–	3
Sistema	Consultar a identificação da aplicação	Positivo	–	5
Sistema	Consultar a saúde da aplicação	Positivo	–	4

Exemplo: regra central do negócio

Com meta de 500 kWh, nenhuma medição passa da meta sozinha, mas a soma do dia chega a 550 kWh. Outros cenários cobrem os limites: consumo igual à meta e medições em dias diferentes não geram alerta.

`medicoes.feature (trecho)`

```
Cenário: Consumo acumulado do dia acima da meta gera alerta
  Dado que foi registrada uma medição de 300.00 kWh em "2026-10-01T08:00:00"
  Quando registro uma medição de 250.00 kWh em "2026-10-01T18:00:00"
  Então o status da resposta deve ser 201
  E a resposta deve respeitar o contrato "medicao.json"
  E o campo "alertaGerado" deve ser "true"
```

4. Testes de API e contrato

Endpoint	Status testados	Contrato
`POST /api/edificios`	201, 400	edificio.json, erro.json
`GET /api/edificios`	200	edificio-lista.json
`GET /api/edificios/{id}`	200, 400, 404	edificio.json, erro.json
`POST /api/edificios/{id}/medicoes`	201, 400, 404	medicao.json, erro.json
`GET /api/edificios/{id}/medicoes`	200, 404	medicao-lista.json, erro.json
`GET /api/alertas`	200	alerta-lista.json
`PATCH /api/alertas/{id}/resolver`	200, 404	alerta.json, erro.json
`GET /api/indicadores`	200	indicadores.json
`GET /api/info`	200	info.json
`GET /actuator/health`	200	health.json

Os erros 400 de JSON malformado e de identificador não numérico são tratados pelo Spring em formato próprio, por isso esses dois cenários validam apenas o status.

Como os contratos foram definidos

Os schemas exigem todos os campos, verificam tipos e formatos e, exceto no health check, proíbem campos não previstos. Assim, remover, renomear ou adicionar um campo reprova o teste. O health check aceita campos extras porque seus detalhes mudam conforme o ambiente.

`src/test/resources/schemas/medicao.json (campos em uma linha para leitura)`

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "title": "Medicao",
  "type": "object",
  "properties": {
    "id": {"type": "integer", "minimum": 1},
    "edificioId": {"type": "integer", "minimum": 1},
    "kwh": {"type": "number", "minimum": 0},
    "dataHora": {"type": "string", "pattern": "^\\d{4}-\\d{2}-\\d{2}T\\d{2}:\\d{2}:\\d{2}(\\.\\d+)?$"},
    "alertaGerado": {"type": "boolean"}
  },
  "required": ["id", "edificioId", "kwh", "dataHora", "alertaGerado"],
  "additionalProperties": false
}
```

Além disso, `ContratoJsonSchemaTest` tem 5 testes que conferem o próprio contrato: um aprova uma resposta correta, e quatro confirmam a reprovação de campo ausente, tipo errado, campo extra e item inválido em lista.

5. Configuração

Dependências de teste

pom.xml (trecho)

```
<!-- BDD: cenarios em Gherkin executados pelo Cucumber sobre JUnit 5 -->
        <dependency>
            <groupId>io.cucumber</groupId>
            <artifactId>cucumber-java</artifactId>
            <version>${cucumber.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>io.cucumber</groupId>
            <artifactId>cucumber-junit-platform-engine</artifactId>
            <version>${cucumber.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>io.cucumber</groupId>
            <artifactId>cucumber-picocontainer</artifactId>
            <version>${cucumber.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.junit.platform</groupId>
            <artifactId>junit-platform-suite</artifactId>
            <scope>test</scope>
        </dependency>

        <!-- Testes de API e de contrato (JSON Schema) -->
        <dependency>
            <groupId>io.rest-assured</groupId>
            <artifactId>rest-assured</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>io.rest-assured</groupId>
            <artifactId>json-schema-validator</artifactId>
            <scope>test</scope>
        </dependency>
```

Configuração do Cucumber

src/test/resources/junit-platform.properties

```
cucumber.glue=br.com.fiap.esg.bdd
cucumber.plugin=pretty,html:target/cucumber-reports/cucumber.html,json:target/cucumber-
reports/cucumber.json,junit:target/cucumber-reports/cucumber.xml
cucumber.publish.quiet=true
cucumber.junit-platform.naming-strategy=long
```

Imagem de testes

Dockerfile.tests

```
# Imagem que executa os testes BDD/API contra a API em execucao (BASE_URL).
FROM maven:3.9-eclipse-temurin-21
WORKDIR /workspace
COPY pom.xml .
RUN mvn -B -q dependency:go-offline
COPY src ./src
ENTRYPOINT ["mvn", "-B", "test", "-Dtest=BddApiTest,ContratoJsonSchemaTest"]
```

Orquestração

docker-compose.test.yml

```
# Sobe API + PostgreSQL e executa os testes contra a aplicacao real.
#   docker compose -f docker-compose.test.yml up --build --abort-on-container-exit --exit-code-from
testes
services:

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: cidades_esg
      POSTGRES_USER: esg
      POSTGRES_PASSWORD: esg_teste
    tmpfs:
      - /var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U esg -d cidades_esg"]
      interval: 5s
      timeout: 5s
      retries: 20

  app:
    build:
      context: .
    environment:
      APP_ENV: test
      DB_URL: jdbc:postgresql://db:5432/cidades_esg
      DB_USER: esg
      DB_PASSWORD: esg_teste
    depends_on:
      db:
        condition: service_healthy

  testes:
    build:
      context: .
      dockerfile: Dockerfile.tests
    environment:
      BASE_URL: http://app:8080
    volumes:
      - ./relatorios-testes:/workspace/target/cucumber-reports
    depends_on:
      app:
        condition: service_healthy
```

Pipeline

.github/workflows/testes-automatizados.yml (jobs de teste)

```
name: Testes automatizados (BDD e API)

on:
  push:
    branches-ignore: [ evidencias-testes ]
  workflow_dispatch:

permissions:
  contents: read

env:
  JAVA_VERSION: '21'

jobs:

  # ------------------------------------------------------------
  # 1) Aplicacao real em porta aleatoria + H2 em memoria
```

```
  # ------------------------------------------------------------
  testes-h2:
    name: Testes - H2
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: ${{ env.JAVA_VERSION }}
          cache: maven

      - name: Executar somente os cenarios negativos (filtro por tag)
        shell: bash
        run: |
          mkdir -p evidencias
          mvn -B test -Dtest=BddApiTest -Dcucumber.filter.tags="@negativo" 2>&1 | tee
evidencias/execucao-negativos.log

      - name: Executar testes (mvn verify)
        shell: bash
        run: |
          mkdir -p evidencias
          mvn -B verify 2>&1 | tee evidencias/execucao-h2.log

      - name: Publicar relatorios
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: relatorios-h2
          path: |
            evidencias/
            target/cucumber-reports/
            target/surefire-reports/

  # ------------------------------------------------------------
  # 2) API e PostgreSQL em containers
  # ------------------------------------------------------------
  testes-docker:
    name: Testes - Docker e PostgreSQL
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Executar testes (Docker Compose)
        shell: bash
        run: |
          mkdir -p evidencias
          bash scripts/run-tests.sh docker 2>&1 | tee evidencias/execucao-docker.log

      - name: Publicar relatorios
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: relatorios-docker
          path: |
            evidencias/
            relatorios-testes/
```

Depois desses dois jobs, o job **Consolidar evidências** gera um resumo da execução e guarda os relatórios na branch `evidencias-testes`. A suíte também faz parte do `mvn verify` do pipeline `ci-cd.yml` existente, que antecede a imagem Docker e os deploys.

6. Como executar

Local, sem Docker (requer JDK 21 e Maven 3.9)

```
bash scripts/run-tests.sh
# Relatório: target/cucumber-reports/cucumber.html
```

Com Docker e PostgreSQL (requer apenas Docker)

```
bash scripts/run-tests.sh docker
# Relatório: relatorios-testes/cucumber.html
```

Somente um grupo de cenários, por tag

```
mvn test -Dtest=BddApiTest -Dcucumber.filter.tags="@negativo"
```

Contra uma API já em execução

```
BASE_URL=http://localhost:8080 mvn test -Dtest=BddApiTest
```

No GitHub Actions, os testes rodam a cada push pelo workflow `testes-automatizados.yml`.

7. Evidências de execução

Execução da entrega no GitHub Actions: **run 41550086327**, commit `109b199`, em 2 de novembro de 2026 (horário de Brasília).
Link: https://github.com/jrpietrafesa/cidades-inteligentes/actions/runs/41550086327

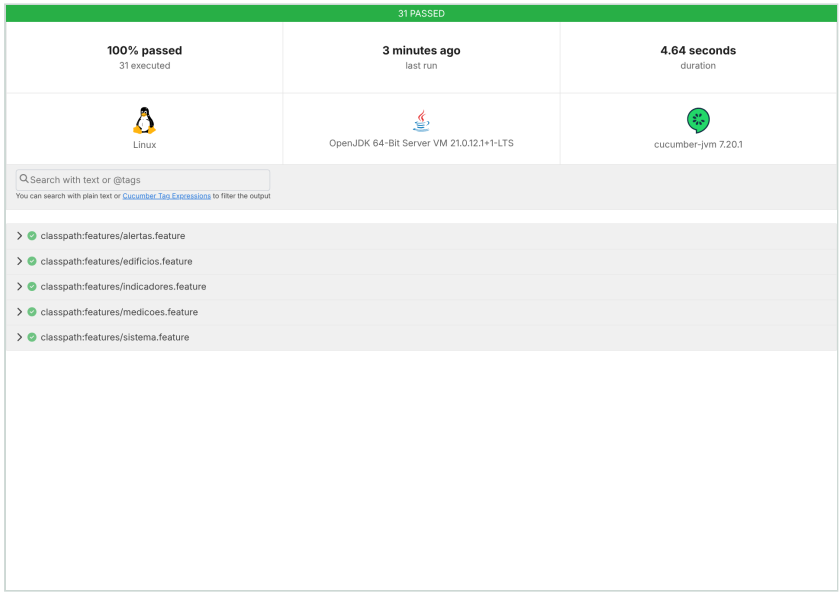
Ambiente	Cenários	Passos	Testes (total)	Falhas
Aplicação + H2	31	142	44	0
Docker + PostgreSQL 16	31	142	36	0

No H2, os 44 testes somam 31 cenários, 5 de contrato, 4 unitários e 4 de integração. No Docker, os 36 somam os cenários e os testes de contrato.

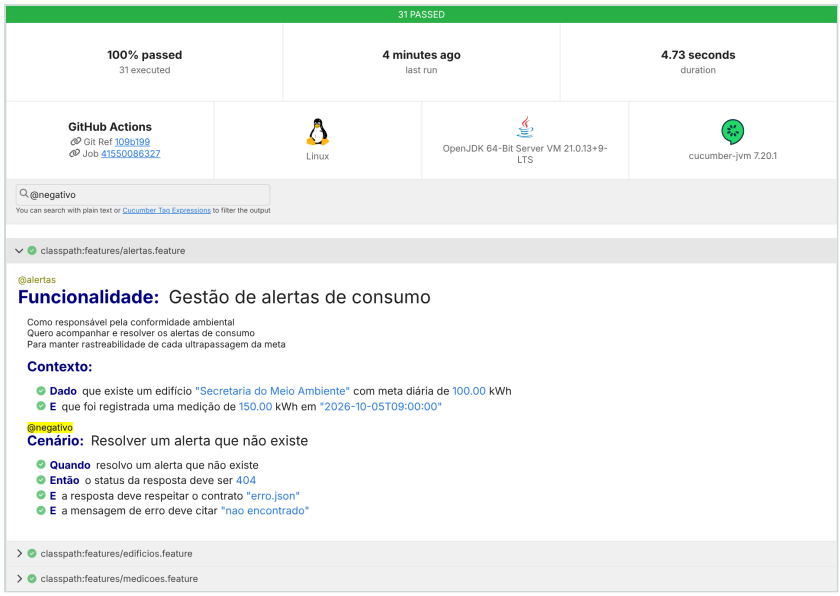
Jobs do pipeline

Job	Resultado	Duração
Testes - Docker e PostgreSQL	Sucesso	89 s
Testes - H2	Sucesso	44 s
Consolidar evidencias	Sucesso	14 s

Dados obtidos pela API do GitHub.



Relatório do Cucumber: execução com Docker e PostgreSQL, 31 cenários aprovados



Relatório do Cucumber: cenários negativos filtrados pela tag @negativo

Log: Docker e PostgreSQL

execucao-docker.log (linhas selecionadas)

```
db-1  | waiting for server to start....2026-11-03 00:04:49.790 UTC [43] LOG:  starting PostgreSQL 16.15
on x86_64-pc-l
db-1  | 2026-11-03 00:04:50.044 UTC [1] LOG:  starting PostgreSQL 16.15 on x86_64-pc-linux-musl,
compiled by gcc (Alpi
app-1  | 2026-11-02T21:05:01.554-03:00  INFO 1 --- [cidades-esg-inteligentes] [           main]
br.com.fiap.esg.Cidade
testes-1  | [INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.367 s -- in
br.com.fiap.esg.bdd.C
testes-1  | [INFO] Tests run: 31, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.915 s -- in
br.com.fiap.esg.bdd.
testes-1  | [INFO] Tests run: 36, Failures: 0, Errors: 0, Skipped: 0
testes-1  | [INFO] BUILD SUCCESS
```

Log: H2

execucao-h2.log (linhas selecionadas)

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.421 s -- in
br.com.fiap.esg.bdd.ContratoJsonS
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.259 s -- in
br.com.fiap.esg.service.Monitoram
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 5.813 s -- in
br.com.fiap.esg.controller.Cidade
[INFO] Tests run: 31, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.979 s -- in
br.com.fiap.esg.bdd.BddApiTest
[INFO] Tests run: 44, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Log: filtro por tag

Com `@negativo`, rodaram 15 cenários e os outros 16 foram ignorados.

execucao-negativos.log (linhas selecionadas)

```
[WARNING] Tests run: 31, Failures: 0, Errors: 0, Skipped: 16
[INFO] BUILD SUCCESS
```

8. Prova de que os testes detectam erros

Testes aprovados só têm valor se também puderem reprovar. Por isso, a aplicação recebeu dois defeitos propositais em uma branch temporária, usada apenas para esta verificação. Os defeitos não fazem parte da entrega.

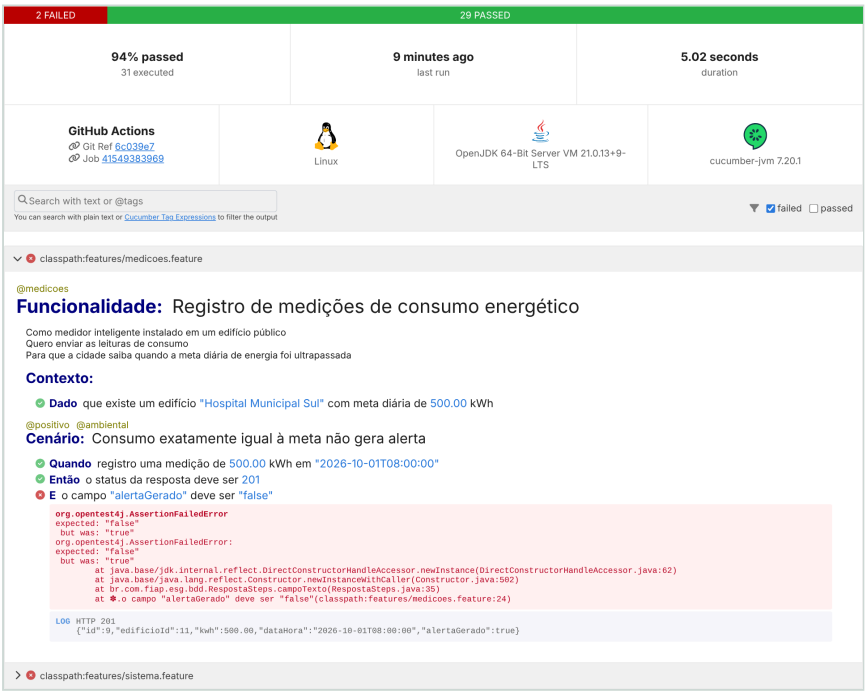
Defeito introduzido	Tipo	Cenário que reprovou	Motivo
Regra da meta alterada de `>` para `>=`	Lógica de negócio	Consumo exatamente igual à meta não gera alerta	esperado `false`, recebido `true`
Campo `versao` removido de `/api/info`	Quebra de contrato	Consultar a identificação da aplicação	JSON Schema: campo obrigatório ausente

Resultado: **run 41549383969**, commit `6c039e7`. Os dois defeitos foram detectados nos dois ambientes, com 2 cenários reprovados e 29 aprovados em cada um, e o pipeline falhou.

Job	Resultado	Duração
Testes - Docker e PostgreSQL	Falha	110 s
Testes - H2	Falha	51 s
Consolidar evidencias	Sucesso	11 s

execucao-com-defeitos.log (linhas selecionadas)

```
expected: "false"
 but was: "true"
error: object has missing required properties (["versao"])
    missing: ["versao"]
[ERROR] Tests run: 31, Failures: 2, Errors: 0, Skipped: 0, Time elapsed: 5.372 s <<< FAILURE! -- in
br.com.fiap.esg.bd
[ERROR] Tests run: 44, Failures: 2, Errors: 0, Skipped: 0
[INFO] BUILD FAILURE
```



Relatório do Cucumber da execução com defeitos, com um dos cenários reprovados aberto

9. Pontos de atenção encontrados no código

A análise para escrever os testes revelou dois comportamentos que merecem revisão. Eles não foram alterados nem são fixados pelos testes, pois mudar a aplicação estava fora do escopo.

- **Histórico de medições:** `GET /api/edificios/{id}/medicoes` sempre retorna `alertaGerado: false`, mesmo quando a medição gerou alerta. Esse valor não é gravado no banco.
- **Alerta resolvido:** se chegar uma nova medição no mesmo dia, o consumo do alerta é atualizado, mas ele continua como `RESOLVIDO`. O consumo adicional não reabre o alerta.

10. Arquivos da entrega

Arquivo ou pasta	Conteúdo
`src/test/`	Cenários Gherkin, passos Java, contratos e testes
`Dockerfile.tests` e `docker-compose.test.yml`	Execução dos testes em containers
`scripts/run-tests.sh`	Comando único de execução, local ou com Docker
`.github/workflows/testes-automatizados.yml`	Pipeline de testes
`README.md`	Instruções para configurar e executar os testes
`docs/evidencias-testes/`	Relatórios HTML do Cucumber e logs das execuções
`docs/Documentacao-Testes-Automatizados.pdf`	Este documento

Os relatórios HTML podem ser abertos em qualquer navegador e mostram cada cenário e cada passo executado.

Apêndice: cenários em Gherkin

Texto integral dos arquivos executados.

src/test/resources/features/edificios.feature

```
# language: pt
@edificios
Funcionalidade: Cadastro de edifícios públicos monitorados
  Como gestor de sustentabilidade da cidade
  Quero cadastrar edifícios com sua meta diária de consumo
  Para acompanhar o compromisso ambiental de cada prédio público

  @positivo @ambiental
  Cenário: Cadastrar edifício com meta ESG válida
    Quando cadastro o edifício "Hospital Municipal Norte" do tipo "HOSPITAL" em "Sao Paulo" com meta
diária de 500.00 kWh
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "edificio.json"
    E o campo "id" deve estar preenchido
    E o campo "nome" deve ser "Hospital Municipal Norte"
    E o campo "tipo" deve ser "HOSPITAL"
    E o campo "limiteDiarioKwh" deve valer 500.00

  @positivo
  Cenário: Consultar um edifício cadastrado
    Dado que existe um edifício "EMEF Jardim Verde" com meta diária de 100.00 kWh
    Quando consulto o edifício cadastrado
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "edificio.json"
    E o campo "nome" deve ser "EMEF Jardim Verde"

  @positivo
  Cenário: Listar os edifícios monitorados
    Dado que existe um edifício "Prefeitura Central" com meta diária de 300.00 kWh
    Quando listo os edifícios
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "edificio-lista.json"
    E a lista deve conter o edifício cadastrado

  @negativo @governanca
  Esquema do Cenário: Rejeitar cadastro inválido: <motivo>
    Quando tento cadastrar um edifício com nome "<nome>", cidade "<cidade>", tipo "<tipo>" e meta "
<meta>"
    Então o status da resposta deve ser 400
    E a resposta deve respeitar o contrato "erro.json"
    E a mensagem de erro deve citar "<campo>"

    Exemplos:
      | motivo           | nome    | cidade    | tipo   | meta    | campo           |
      | nome em branco   |         | Sao Paulo | ESCOLA | 100     | nome            |
      | cidade em branco | Escola  |           | ESCOLA | 100     | cidade          |
      | tipo em branco   | Escola  | Sao Paulo |        | 100     | tipo            |
      | meta zerada      | Escola  | Sao Paulo | ESCOLA | 0       | limiteDiarioKwh |
      | meta negativa    | Escola  | Sao Paulo | ESCOLA | -10     | limiteDiarioKwh |
      | meta ausente     | Escola  | Sao Paulo | ESCOLA | ausente | limiteDiarioKwh |

  @negativo
  Cenário: Rejeitar corpo com JSON malformado
    Quando envio um cadastro de edifício com JSON malformado
    Então o status da resposta deve ser 400

  @negativo
  Cenário: Consultar um edifício que não existe
    Quando consulto um edifício que não existe
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "erro.json"
    E a mensagem de erro deve citar "nao encontrado"
```

```
  @negativo
  Cenário: Consultar um edifício com identificador inválido
    Quando consulto um edifício com identificador "abc"
    Então o status da resposta deve ser 400
```

src/test/resources/features/medicoes.feature

```
# language: pt
@medicoes
Funcionalidade: Registro de medições de consumo energético
  Como medidor inteligente instalado em um edifício público
  Quero enviar as leituras de consumo
  Para que a cidade saiba quando a meta diária de energia foi ultrapassada

  Contexto:
    Dado que existe um edifício "Hospital Municipal Sul" com meta diária de 500.00 kWh

  @positivo @ambiental
  Cenário: Consumo dentro da meta não gera alerta
    Quando registro uma medição de 300.00 kWh em "2026-10-01T08:00:00"
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "medicao.json"
    E o campo "kwh" deve valer 300.00
    E o campo "alertaGerado" deve ser "false"
    E o edifício não deve ter alerta aberto

  @positivo @ambiental
  Cenário: Consumo exatamente igual à meta não gera alerta
    Quando registro uma medição de 500.00 kWh em "2026-10-01T08:00:00"
    Então o status da resposta deve ser 201
    E o campo "alertaGerado" deve ser "false"

  @positivo @ambiental
  Cenário: Consumo acumulado do dia acima da meta gera alerta
    Dado que foi registrada uma medição de 300.00 kWh em "2026-10-01T08:00:00"
    Quando registro uma medição de 250.00 kWh em "2026-10-01T18:00:00"
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "medicao.json"
    E o campo "alertaGerado" deve ser "true"

  @positivo @ambiental
  Cenário: Medições de dias diferentes não se somam
    Dado que foi registrada uma medição de 300.00 kWh em "2026-10-01T08:00:00"
    Quando registro uma medição de 250.00 kWh em "2026-10-02T08:00:00"
    Então o status da resposta deve ser 201
    E o campo "alertaGerado" deve ser "false"

  @positivo
  Cenário: Medição sem data usa o horário atual
    Quando registro uma medição de 10.00 kWh sem informar a data
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "medicao.json"
    E o campo "dataHora" deve estar preenchido

  @positivo @governanca
  Cenário: Listar o histórico de medições do edifício
    Dado que foi registrada uma medição de 10.00 kWh em "2026-10-01T08:00:00"
    E que foi registrada uma medição de 20.00 kWh em "2026-10-03T08:00:00"
    E que foi registrada uma medição de 30.00 kWh em "2026-10-02T08:00:00"
    Quando listo as medições do edifício
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "medicao-lista.json"
    E a lista deve ter 3 medições, da mais recente para a mais antiga

  @negativo @governanca
  Esquema do Cenário: Rejeitar medição inválida: <motivo>
    Quando tento registrar uma medição com valor "<valor>"
    Então o status da resposta deve ser 400
```

```
    E a resposta deve respeitar o contrato "erro.json"
    E a mensagem de erro deve citar "kwh"

    Exemplos:
      | motivo         | valor   |
      | consumo zerado | 0       |
      | consumo negativo | -5    |
      | consumo ausente | ausente |

  @negativo
  Cenário: Registrar medição em edifício que não existe
    Quando registro uma medição em um edifício que não existe
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "erro.json"

  @negativo
  Cenário: Listar medições de edifício que não existe
    Quando listo as medições de um edifício que não existe
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "erro.json"
```

src/test/resources/features/alertas.feature

```
# language: pt
@alertas
Funcionalidade: Gestão de alertas de consumo
  Como responsável pela conformidade ambiental
  Quero acompanhar e resolver os alertas de consumo
  Para manter rastreabilidade de cada ultrapassagem da meta

  Contexto:
    Dado que existe um edifício "Secretaria do Meio Ambiente" com meta diária de 100.00 kWh
    E que foi registrada uma medição de 150.00 kWh em "2026-10-05T09:00:00"

  @positivo @governanca
  Cenário: Consultar alerta aberto com dados de rastreabilidade
    Quando consulto os alertas abertos
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "alerta-lista.json"
    E o alerta do edifício deve registrar consumo de 150.00 kWh contra a meta de 100.00 kWh na data
"2026-10-05"

  @positivo @governanca
  Cenário: Nova medição no mesmo dia atualiza o alerta existente
    Dado que foi registrada uma medição de 40.00 kWh em "2026-10-05T15:00:00"
    Quando consulto os alertas abertos
    Então o status da resposta deve ser 200
    E o alerta do edifício deve registrar consumo de 190.00 kWh contra a meta de 100.00 kWh na data
"2026-10-05"

  @positivo @governanca
  Cenário: Resolver um alerta aberto
    Dado que o alerta do edifício foi aberto
    Quando resolvo o alerta do edifício
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "alerta.json"
    E o campo "status" deve ser "RESOLVIDO"
    E o edifício não deve ter alerta aberto

  @negativo
  Cenário: Resolver um alerta que não existe
    Quando resolvo um alerta que não existe
    Então o status da resposta deve ser 404
    E a resposta deve respeitar o contrato "erro.json"
    E a mensagem de erro deve citar "nao encontrado"
```

src/test/resources/features/indicadores.feature

```
# language: pt
@indicadores
Funcionalidade: Indicadores ambientais da cidade
  Como gestor público
  Quero consultar consumo, emissões e alertas consolidados
  Para reportar o desempenho ambiental da cidade

  @positivo @ambiental
  Cenário: Indicadores refletem uma medição acima da meta
    Dado que anotei os indicadores atuais
    E que existe um edifício "Biblioteca Municipal" com meta diária de 100.00 kWh
    Quando registro uma medição de 150.00 kWh em "2026-10-06T10:00:00"
    E consulto os indicadores
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "indicadores.json"
    E o indicador "edificiosMonitorados" deve ter variado 1
    E o indicador "medicoesRegistradas" deve ter variado 1
    E o indicador "consumoTotalKwh" deve ter variado 150.00
    E o indicador "alertasAbertos" deve ter variado 1
    E a emissão estimada deve corresponder ao consumo total pelo fator 0.0385 kgCO2 por kWh

  @positivo @governanca
  Cenário: Resolver alerta atualiza os indicadores
    Dado que existe um edifício "Centro Cultural" com meta diária de 100.00 kWh
    E que foi registrada uma medição de 150.00 kWh em "2026-10-06T10:00:00"
    E que anotei os indicadores atuais
    E que o alerta do edifício foi resolvido
    Quando consulto os indicadores
    Então o status da resposta deve ser 200
    E o indicador "alertasAbertos" deve ter variado -1
    E o indicador "alertasResolvidos" deve ter variado 1
```

src/test/resources/features/sistema.feature

```
# language: pt
@sistema
Funcionalidade: Identificação e saúde da aplicação
  Como operador do ambiente
  Quero verificar a versão e a saúde da API
  Para confirmar que o serviço está disponível

  @positivo
  Cenário: Consultar a identificação da aplicação
    Quando consulto a identificação da aplicação
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "info.json"
    E o campo "aplicacao" deve ser "Cidades ESG Inteligentes"
    E o campo "ambiente" deve ser "test"

  @positivo
  Cenário: Consultar a saúde da aplicação
    Quando consulto a saúde da aplicação
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "health.json"
    E o campo "status" deve ser "UP"
```